

ARQUITETURA DE SOFTWARE — UNIDADE 1

Estudo de caso: plataforma hospitalar

Cenários, matriz de estilos, estrutura inicial e ADR-001

Exercício	Foco	Entregável
1	Cenários mensuráveis por capacidade	cenarios.md
2	Matriz de estilos por capacidade	matriz-estilos.md
3	Estrutura inicial e diagrama	estrutura.md
4	Consequências e decisão registrada	ADR-001-estrutura-inicial.md

Pasta: entregas/unidade-1/estudo-de-caso/ · 22 de agosto de 2026

Exercício 1 — Cenários mensuráveis por capacidade

Plataforma hospitalar administrativa — Unidade 1, Estudo de caso.

Forças dominantes registradas em uma palavra:

Capacidade	Força dominante
Agenda	consistência
Triagem administrativa	extensibilidade
Faturamento	vazão (throughput)

Todos os valores numéricos abaixo são **suposições iniciais**, marcadas como tal. Servem para tornar as alternativas comparáveis, não são compromisso de produção.

Cenário 1 — Agenda (consistência)

- **Fonte do estímulo:** duas recepcionistas de unidades diferentes, em terminais distintos
- **Estímulo:** 50 solicitações de reserva do mesmo horário do mesmo profissional, dentro de 1 segundo
- **Ambiente:** operação normal, sob carga de 100 reservas/s
- **Artefato:** módulo Agenda (reserva e confirmação)
- **Resposta:** exatamente uma reserva confirmada com protocolo; as demais recebem recusa explícita por conflito, com o protocolo da reserva vencedora
- **Medida:** o reservas duplicadas em 1.000 tentativas concorrentes; p95 de confirmação ou recusa ≤ 300 ms (*suposição*)

Coleta: script de carga (locust ou k6) dispara as tentativas concorrentes em ambiente de laboratório; a contagem de duplicatas sai de uma consulta GROUP BY (profissional, horário) HAVING COUNT(*) > 1 na base, e o p95 do relatório da ferramenta. Coletado pela dupla, no laboratório da disciplina, com a massa sintética versionada junto do teste.

Cenário 2 — Triagem administrativa (extensibilidade)

- **Fonte do estímulo:** equipe de TI da unidade Norte, atendendo a uma exigência administrativa local
- **Estímulo:** pedido para acrescentar uma etapa nova de coleta (questionário administrativo da unidade) ao fluxo de pré-atendimento
- **Ambiente:** período de evolução, sem indisponibilidade programada
- **Artefato:** módulo Triagem (núcleo + extensões) e seu contrato de plugin
- **Resposta:** a etapa entra como extensão registrada pelo contrato, é ligada e desligada por configuração por unidade, e o núcleo segue publicando os mesmos fatos auditáveis

- **Medida:** o arquivos do núcleo alterados no diff; 100% dos testes do núcleo passando sem edição; ativação/desativação por configuração, sem novo build (*suposição*)

Coleta: `git diff --stat` da branch da extensão, conferindo que nenhum caminho sob `triagem/nucleo/` aparece; execução da suíte do núcleo antes e depois; um teste de integração liga e desliga a extensão só por variável de configuração. Coletado pela dupla, revisado por outra dupla na aula.

Cenário 3 — Faturamento (vazão / throughput)

- **Fonte do estímulo:** rotina de fechamento diário, disparada por agendador
- **Estímulo:** lote de 10.000 registros administrativos vindos de três origens distintas
- **Ambiente:** operação normal em janela noturna de fechamento
- **Artefato:** módulo Faturamento (validação → normalização → correlação → saída por parceiro)
- **Resposta:** o lote é validado, normalizado, correlacionado com as autorizações e entregue por parceiro; cada rejeição sai identificada por etapa, motivo e chave de correlação, sem descarte silencioso
- **Medida:** ≥ 500 registros/s de ponta a ponta; lote de 10.000 concluído em ≤ 20 s; 100% das rejeições com etapa e motivo preenchidos (*suposição*)

Coleta: execução com massa sintética gerada por script (`gerar_massa.py`, semente fixa), em ambiente registrado (CPU, memória, versão); o próprio pipeline emite ao final registros/s, total processado e total rejeitado por etapa. Coletado pela dupla, com o log do run anexado à entrega para que outra pessoa repita.

Nota de escopo

As três capacidades foram entregues completas. As demais capacidades do caso (cadastro, elegibilidade, autorização, exames e resultados, notificações, auditoria) não receberam cenário próprio nesta unidade; cadastro, elegibilidade e autorização aparecem indiretamente dentro da triagem administrativa.

arquivo: `entregas/unidade-1/estudo-de-caso/cenarios.md`

Exercício 2 — Matriz de estilos por capacidade

Régua: os três cenários do [Exercício 1](#). Cada célula diz **o que o estilo faz por aquela capacidade e a que custo**.

Estilo	Agenda (consistência)	Triagem administrativa (extensibilidade)	Faturamento (vazão)
Camadas	separa entrada, aplicação, regra e persistência, deixando a reserva atômica confinada na camada de persistência; ao custo de cada regra nova atravessar quatro camadas para chegar ao banco	separa coleta, regra e integração em níveis; ao custo de uma etapa nova por unidade tocar as três camadas, o que contraria o cenário 2 (o arquivos do núcleo alterados)	permite testar validação e normalização sem infraestrutura; ao custo de o fluxo em lote não ter forma natural de expressão em níveis, exigindo um orquestrador improvisado na camada de aplicação
Pipes and filters	pouco natural para reserva interativa: a decisão "confirma ou recusa" é uma transação única, não uma sequência de transformações; ao custo de espalhar o estado da reserva entre filtros e perder a atomicidade do cenário 1	organiza bem as etapas lineares (identificar → verificar elegibilidade → autorizar); ao custo de etapas opcionais por unidade virarem desvios condicionais dentro do tubo, e de o retorno síncrono ao balcão ficar amarrado ao filtro mais lento	corresponde diretamente ao cenário 3: cada filtro faz uma coisa, gera resultado explícito e permite medir vazão por etapa; ao custo de manter contrato e correlação entre filtros para que a rejeição saiba de onde veio
Microkernel	útil apenas se as regras de reserva variarem muito por unidade; hoje não variam, e o encaixe de plugins acrescenta indireção sobre um caminho que precisa ser curto e atômico	atende ao cenário 2 diretamente: núcleo com identidade da jornada, estados permitidos e emissão de fatos; extensões ligadas por configuração; ao custo de um contrato de plugin que precisa ser estável e de plugins sem acesso ao banco do núcleo	pode isolar layouts de parceiro como plugins de saída; ao custo de não organizar o miolo do lote — a vazão continua dependendo de um pipeline dentro do núcleo, então o estilo resolve só a borda
Monólito modular	preserva a consistência local: reserva é transação em um único banco, sem coordenação distribuída, atendendo o cenário 1 com o mecanismo mais simples disponível; ao custo de compartilhar processo e escala com os vizinhos	dá fronteira de módulo com interface explícita e permite trocar a implementação interna sem nova implantação; ao custo de, sozinho, não dizer nada sobre como acrescentar etapas — precisa de um estilo interno (microkernel) para atender o cenário 2	mantém os módulos próximos, com contratos internos e sem serialização de rede entre etapas; ao custo de um lote pesado disputar CPU e conexões com a agenda no mesmo processo, ameaçando o p95 do cenário 1

Células carentes de evidência

- 1. **Monólito modular × Faturamento** — não sabemos se o lote de 10.000 registros degrada o p95 da agenda no mesmo processo. *Medição que resolve:* rodar o cenário 3 (lote de 10.000) e o cenário 1 (100

reservas/s) simultaneamente no mesmo ambiente e comparar o p95 da agenda com o da execução isolada. Se a degradação passar de 300 ms, a célula vira desfavorável.

2. **Microkernel × Triagem administrativa** — não sabemos se as etapas reais das unidades cabem no contrato "recebe contexto mínimo, devolve estado, pendências e evidências", ou se vão querer transação e tabela do núcleo. *Medição que resolve*: implementar uma extensão piloto real (o questionário da unidade Norte) e verificar se ela roda sem alterar arquivo do núcleo e sem acesso direto ao banco. Se precisar de qualquer um dos dois, o limite do plugin está errado.

Qual capacidade separa mais os estilos

A **agenda** é a que mais separa. Faturamento aceita bem tanto pipes and filters quanto camadas — os dois conseguem descrever validação e transformação, e a diferença entre eles é de conveniência, não de viabilidade. Já a reserva concorrente elimina candidatos: pipes and filters fragmenta a decisão que precisa ser atômica, e microkernel acrescenta indireção sem resolver o conflito. Sobra consistência local, que camadas dentro de um monólito modular entrega com o mecanismo mais simples: uma transação em um banco só.

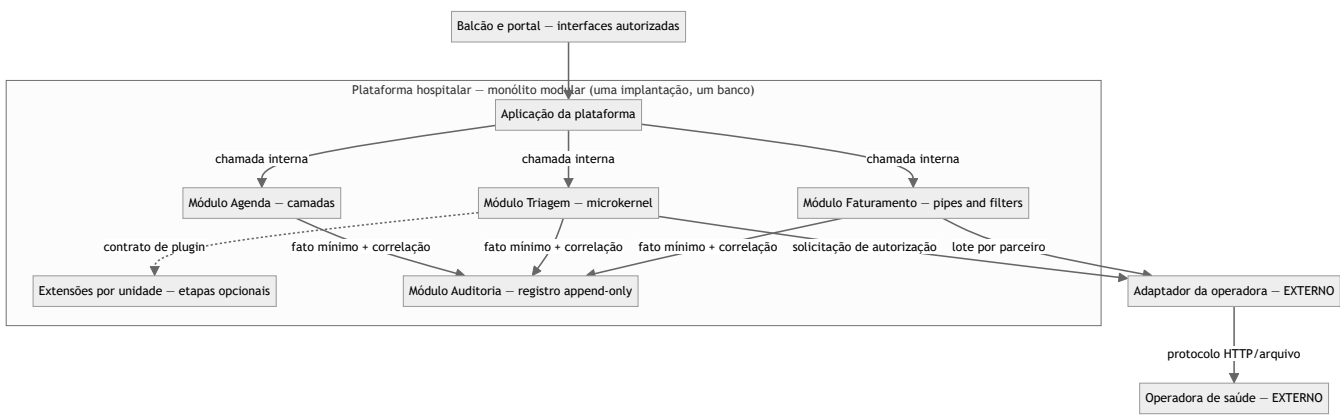
arquivo: `entregas/unidade-1/estudo-de-caso/matriz-estilos.md`

Exercício 3 — Estrutura inicial e diagrama

Estrutura escolhida

Uma implantação única — **monólito modular** — com os módulos Agenda, Triagem, Faturamento e Auditoria, cada um com interface explícita e modelo de dados próprio. Dentro de cada módulo, o estilo interno é escolhido pela força que domina aquela capacidade: camadas na Agenda, microkernel na Triagem, pipes and filters no Faturamento. O que fala com a operadora fica atrás de um adaptador, fora da fronteira da plataforma.

Diagrama



Texto alternativo: diagrama de blocos em que o balcão e o portal, como interfaces autorizadas, levam à Aplicação da plataforma; dentro do quadro "Plataforma hospitalar — monólito modular", a aplicação encaminha por chamada interna para os módulos Agenda (camadas), Triagem (microkernel) e Faturamento (pipes and filters); a Triagem liga extensões por unidade através de um contrato de plugin, e os três módulos gravam fatos mínimos com correlação no módulo Auditoria; fora do quadro, Triagem e Faturamento conversam com o Adaptador da operadora, que por sua vez fala com a Operadora de saúde — ambos marcados como externos.

Figura 1 — Monólito modular da plataforma hospitalar, com estilo interno por capacidade.
Fonte: elaborado pela dupla.

Leitura textual da figura: as interfaces autorizadas do balcão e do portal chegam à Aplicação da plataforma. Dentro do quadro da plataforma, uma única implantação sobre um único banco, a aplicação encaminha a requisição por chamada interna para um dos três módulos: Agenda, organizada em camadas; Triagem, organizada como microkernel; ou Faturamento, organizado como pipes and filters. A Triagem se conecta às extensões por unidade por um contrato de plugin, representado por linha tracejada porque a ligação é opcional e configurável. Agenda, Triagem e Faturamento enviam ao módulo Auditoria um fato mínimo acompanhado da chave de correlação — não o conteúdo clínico, não o controle da regra. Saindo do quadro, Triagem envia solicitações de autorização e Faturamento envia lotes por parceiro para o Adaptador da operadora, que traduz o modelo da plataforma para o protocolo da Operadora de saúde. Adaptador e operadora estão fora da fronteira da plataforma e marcados como externos.

Módulo, estilo interno e cenário atendido

- **Agenda — camadas — cenário 1 (consistência).** As camadas separam entrada, aplicação, regra e persistência, e a reserva vira uma transação local em um único banco; é isso que garante uma confirmação e 49 conflitos explícitos em 50 tentativas simultâneas, sem coordenar serviços.
- **Triagem — microkernel — cenário 2 (extensibilidade).** O núcleo guarda identidade da jornada, estados permitidos e emissão de fatos; a etapa nova da unidade Norte entra como extensão pelo contrato, ligada por configuração, com zero arquivo do núcleo alterado.
- **Faturamento — pipes and filters — cenário 3 (vazão).** Validação, normalização, correlação e saída por parceiro são filtros com resultado explícito, o que permite medir registros/s por etapa e devolver cada rejeição com etapa, motivo e correlação.

O módulo Auditoria não tem estilo interno próprio nesta versão: é um registro append-only que recebe fatos mínimos dos três módulos e sustenta a força de rastreabilidade declarada no contexto compartilhado. O diagrama **não** significa que todo módulo enxerga todo dado — cada módulo controla seu modelo e só é alcançado pela sua interface.

arquivo: `entregas/unidade-1/estudo-de-caso/estrutura.md`

ADR-001 — Estrutura inicial da plataforma hospitalar

- **Estado:** aceito (provisório — revisável pelos gatilhos abaixo)
- **Data:** 2026-08-22
- **Decisores:** dupla do estudo de caso, Unidade 1
- **Escopo:** estrutura geral da plataforma administrativa hospitalar e estilos internos por capacidade

Contexto

A plataforma coordena a jornada administrativa do paciente — cadastro, agenda, elegibilidade, autorização, exames, faturamento, notificações e auditoria — sem recomendar tratamento nem interpretar resultados. Três capacidades puxam para lados opostos e foram usadas como régua ([cenários](#)): a agenda exige consistência sob concorrência, a triagem administrativa exige extensibilidade por unidade, e o faturamento exige vazão em lote.

Não há hoje medição de carga real, nem equipe dividida por capacidade, nem exigência declarada de cadência de implantação independente. A decisão é tomada com essa ausência de evidência explícita.

Forças

Força	Onde aperta	Cenário
Consistência sob concorrência	reserva do mesmo horário por dois terminais	1
Extensibilidade	etapa administrativa nova por unidade, sem mexer no núcleo	2
Vazão (throughput)	lote diário de dezenas de milhares de registros	3
Rastreabilidade	reconstruir quem fez o quê e quando	contexto compartilhado

Alternativas consideradas

- **Aplicação única sem módulos** atenderia simplicidade operacional e velocidade inicial, mas traria acoplamento sem fronteira: uma regra de faturamento poderia ler e escrever o modelo da agenda. Foi descartada porque não há como verificar o cenário 2 (zero arquivos do núcleo alterados) quando não existe núcleo delimitado.
- **Microkernel como estrutura global** atenderia extensibilidade em toda a plataforma, mas traria um contrato de plugin genérico demais para caber tanto numa reserva atômica quanto num lote de 10.000 registros. Foi descartada porque a evidência disponível mostra que só a triagem tem variação por unidade — as outras duas capacidades pagariam a indireção sem receber o benefício.
- **Implantação independente por capacidade (um serviço por capacidade)** atenderia escala e falha isoladas, exatamente o custo que estamos aceitando abaixo, mas traria coordenação distribuída na reserva concorrente: garantir "nunca duas reservas no mesmo horário" passaria a exigir protocolo entre serviços em vez de uma transação de banco. Foi descartada pela ausência de evidência — não há medição que mostre que a agenda ou o faturamento precisem escalar separadamente hoje.

- **Pipes and filters como estrutura global** atenderia a vazão do faturamento, mas traria fragmentação da decisão de reserva entre filtros. Foi descartada porque o cenário 1 exige atomicidade, e um pipeline distribui o estado justamente onde ele precisa ser único.

Decisão

Adotamos **monólito modular** como estrutura inicial: uma implantação, um banco, módulos Agenda, Triage, Faturamento e Auditoria com interfaces explícitas e modelo de dados próprio. Dentro dos módulos, o estilo segue a força dominante — camadas na Agenda, microkernel na Triage, pipes and filters no Faturamento. Acesso à operadora fica atrás de um adaptador externo. A estrutura completa está em [estrutura.md](#).

Consequências

Favorável: garantir "nunca duas reservas no mesmo horário" é simples, porque a reserva é uma transação local em um único banco — sem coordenar serviços distribuídos. Observável no cenário 1: o duplicadas em 1.000 tentativas concorrentes, medido na própria base.

Favorável: a triagem ganha etapas novas por configuração, sem nova implantação da plataforma. Observável no diff da extensão piloto: o arquivos do núcleo alterados.

Favorável: a auditoria recebe fatos mínimos com correlação dos três módulos no mesmo processo, sem perda de ordem entre serviços.

Desfavorável (aceita): enquanto tudo estiver numa só implantação, agenda e faturamento dividem processo, escala e falha; um lote pesado disputa CPU e conexões com a reserva. Aceitamos esse custo enquanto o p95 da agenda permanecer dentro de 300 ms durante a janela de fechamento. Se o faturamento precisar escalar sozinho, o módulo terá de ser extraído.

Desfavorável (aceita): fronteira de módulo é convenção, não barreira física. Aceitamos verificar as dependências internas por teste de fronteira enquanto não houver separação de processo.

Desfavorável (aceita): um bug de memória em qualquer módulo derruba os demais. Aceitamos esse raio de falha enquanto não houver exigência declarada de disponibilidade por capacidade.

Evidências

Evidência	Onde se reproduz
Cenário 1 — o duplicadas em 1.000 tentativas concorrentes	script de carga + consulta <code>GROUP BY (profissional, horário) HAVING COUNT(*) > 1</code> , laboratório da disciplina
Cenário 2 — extensão piloto da triagem	<code>git diff --stat</code> da branch da extensão + suíte do núcleo sem edição
Cenário 3 — lote sintético de 10.000 registros	<code>gerar_massa.py</code> com semente fixa; o pipeline reporta registros/s e rejeições por etapa; ambiente registrado no log anexo
Teste de fronteira entre módulos	verificação de imports/dependências entre pacotes (a escrever)
Revisão do diagrama por outra dupla	leitura textual conferida contra a figura em aula

As três primeiras evidências ainda não foram executadas; os valores do [Exercício 1](#) estão marcados como suposições.

Gatilho de revisão

Este registro é reaberto se qualquer uma destas observações ocorrer:

1. Se o p95 de confirmação da agenda ultrapassar **500 ms** durante a janela de fechamento do faturamento, em duas medições distintas na mesma semana.
2. Se o lote diário deixar de fechar em $\leq 5 \text{ min}$ por três dias consecutivos com a instância atual, indicando necessidade comprovada de escala própria do faturamento.
3. Se uma extensão da triagem precisar alterar arquivo do núcleo ou acessar diretamente o banco do núcleo — o limite do plugin está errado e o estilo interno precisa ser revisto.
4. Se surgir exigência declarada de cadência de implantação independente entre duas capacidades (equipes separadas liberando em ritmos diferentes).

Campos que dependem de execução

Nenhum campo do template ficou vazio, mas as medidas dos gatilhos 1 e 2 herdam os valores supostos do Exercício 1. Ao primeiro run real de laboratório, esses números devem ser confirmados ou corrigidos, e este ADR atualizado com a data da medição.

arquivo: `entregas/unidade-1/estudo-de-caso/ADR-001-estrutura-inicial.md`